

MES COLLEGE OF ENGINEERING, KUTTIPPURAM

Department of Artificial Intelligence and Data Science

Affiliated to APJ Abdul Kalam Technological University

MINI PROJECT REPORT

on

PERFORMANCE EVALUATION OF CPI FOR DIFFERENT INSTRUCTION MIXES IN A RISC PROCESSOR

In partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Artificial Intelligence and Data Science

Submitted by

Name of Student	Register Number
FATHIMA MINSHA	MES24AD019
FARHA FATHIMA	MES24AD014
FATHIMA HAFNA SHERIN	MES24AD018
FATHIMATH NAJA	MES24AD023
HASNA TK	MES24AD025
RIDHIN MUHAMMED	MES24AD039

Submitted to

SHERIK K. K.

Assistant Professor, Dept. of AI & Data Science

MES College of Engineering, Kuttippuram

2025 – 2026

Abstract

Processor performance is fundamentally shaped by how efficiently a pipelined CPU executes instructions. This project evaluates the performance of a 5-stage pipelined RISC-V processor by measuring Cycles Per Instruction (CPI) across three distinct instruction mixes: arithmetic-heavy, memory-heavy, and control-heavy programs, each designed to isolate a specific type of pipeline behavior.

Three custom RISC-V assembly programs were written and executed using the Ripes RISC-V visual simulator on a 5-stage pipeline model (IF, ID, EX, MEM, WB). The arithmetic-heavy program employs exclusively register-to-register operations (ADDI, ADD, SUB, MUL, DIV) to benchmark near-ideal pipeline throughput. The memory-heavy program repeatedly performs LW and SW instructions on a data array to expose load-use hazards and memory latency. The control-heavy program chains multiple conditional branch instructions (BEQ, BNE, BLT, BGE) to demonstrate the performance cost of pipeline flushing due to control hazards.

Performance metrics — total clock cycles, instructions retired, CPI, and IPC — were captured directly from the Ripes Execution Info panel after each program ran to completion. Results confirm that arithmetic-heavy programs achieve the lowest CPI of 1.25 (IPC = 0.800), memory-heavy programs show moderate degradation at CPI 1.62 (IPC = 0.619), and control-heavy programs exhibit the highest CPI of 2.33 (IPC = 0.429). The experiment confirms the well-established ordering: Arithmetic CPI < Memory CPI < Control CPI, and provides quantitative evidence that instruction mix is a primary determinant of processor efficiency in a pipelined RISC-V architecture.

Table of Contents

Abstract	2
List of Figures	3
List of Tables	3
1. Introduction	4
2. Literature Review	5
3. System Design	6
4. Experimental Methodology	8
5. Program Descriptions and Implementation	9
6. Experimental Results	13
7. Result Analysis and Discussion	14
8. CPI Comparison	15
9. Advantages of the Experiment	16
10. Limitations	16
11. Applications	17
12. Future Enhancements	17
13. Conclusion	18
14. References	18

List of Figures

- Fig 1 – RISC-V Assembly Code: Arithmetic-Heavy Instruction Mix (Ripes Source Editor)
- Fig 2 – Ripes 5-Stage Processor Simulation: Arithmetic-Heavy Program (Cycles=20, CPI=1.25, IPC=0.8)
- Fig 3 – RISC-V Assembly Code: Memory-Heavy Instruction Mix (.data + LW/SW)
- Fig 4 – Ripes 5-Stage Processor Simulation: Memory-Heavy Program (Cycles=21, CPI=1.62, IPC=0.619)
- Fig 5 – RISC-V Assembly Code: Control-Heavy Instruction Mix (BEQ, BNE, BLT, BGE)
- Fig 6 – Ripes 5-Stage Processor Simulation: Control-Heavy Program (Cycles=21, CPI=2.33, IPC=0.429)

List of Tables

- Table 1 – Pipeline Stages Description
- Table 2 – Instruction Mix vs. CPI Effect
- Table 3 – Simulation Configuration
- Table 4 – Experimental Results: CPI Comparison Across All Programs
- Table 5 – Detailed Performance Metrics from Ripes Simulation
- Table 6 – Hazard Types and Their Impact on Each Program

1. Introduction

The performance of a processor is strongly influenced by how efficiently it executes instructions. Modern processors improve performance using instruction pipelining, a technique where multiple instructions are overlapped during execution by dividing processing into stages. While pipelining increases throughput, its efficiency depends heavily on the nature of instructions being executed.

One of the most important metrics used to evaluate processor performance is Cycles Per Instruction (CPI). CPI indicates the average number of clock cycles required to execute one instruction. In an ideal pipelined processor without delays, CPI approaches 1. However, practical systems experience delays due to hazards, memory latency, and control-flow changes that all increase CPI above the theoretical minimum.

Different programs contain different proportions of arithmetic, memory, and control-flow instructions. This variation is known as the instruction mix. Each instruction type interacts differently with the pipeline: arithmetic instructions execute quickly and smoothly, memory instructions introduce latency waiting for data memory responses, and branch instructions can cause pipeline flushes when the processor must discard incorrectly fetched instructions. Understanding how instruction mix affects CPI is fundamental to processor design, compiler optimization, and high-performance software development.

This project evaluates how CPI varies with different instruction mixes using three custom RISC-V assembly programs executed on the Ripes 5-stage pipelined processor simulator. By isolating each instruction type and measuring CPI directly from the simulator, the project provides clear empirical evidence of the relationship between instruction mix and processor efficiency.

2. Literature Review

2.1 Processor Architecture Background

The concept of instruction pipelining was introduced in early IBM mainframe systems and popularized with RISC (Reduced Instruction Set Computer) architectures in the 1980s. Patterson and Hennessy formalized the 5-stage RISC pipeline model in their landmark textbook "Computer Organization and Design," which remains the standard reference in computer architecture education worldwide. The key insight of RISC pipelining is that simple, fixed-length instructions can be overlapped across multiple functional stages, dramatically increasing throughput without requiring faster clock speeds.

RISC-V, an open-source Instruction Set Architecture developed at the University of California, Berkeley, has emerged as a modern platform for computer architecture education and research. Its clean, modular design makes it ideal for pipeline simulation and performance analysis. Unlike proprietary ISAs, RISC-V is freely available and has attracted widespread adoption in both academic and industrial settings.

2.2 CPI and Instruction Mix

Research in processor performance has consistently demonstrated that CPI is highly sensitive to instruction mix. The foundational formula $CPI = \text{Base CPI} + \text{Memory Stall Cycles per Instruction}$ (Hennessy & Patterson) captures how different instruction types contribute differently to overall cycle count. Arithmetic instructions, which complete entirely within the ALU, approach the ideal CPI of 1.0. Load/store instructions introduce at least one additional stall cycle per load due to memory latency. Branch instructions introduce a 1-2 cycle flush penalty each time the branch is taken and the fetched instructions must be discarded.

Studies in compiler optimization have shown that instruction scheduling — reordering instructions to avoid hazards without changing program semantics — can reduce effective CPI by 10-30% in typical programs. This underlines the practical importance of understanding how instruction type affects pipeline behavior.

2.3 Pipeline Hazards

Pipeline hazards are conditions that prevent the next instruction from executing in its designated clock cycle. Three types are recognized in literature. Structural hazards occur when hardware resources are insufficient to support simultaneous use by multiple pipeline stages; in the Ripes 5-stage model these are avoided by design through separate instruction and data memories. Data hazards (specifically RAW — Read After Write) occur when an instruction needs a register value that a preceding instruction has not yet produced; these are mitigated by data forwarding. Control hazards arise from branch instructions where the correct next instruction is not known until the branch condition is evaluated in the EX stage, requiring a 1-2 cycle flush penalty when a branch is taken.

2.4 Ripes Simulator

Ripes is an open-source, graphical RISC-V processor simulator that visually shows instruction movement through pipeline stages in real time. It supports multiple processor models including a 5-stage pipeline, provides register file views, memory inspection, and an Execution Info panel showing live performance metrics including Cycles, Instructions Retired, CPI, and IPC. It was selected for this project because of its clear visual representation of pipeline behavior and the accuracy of its performance measurements, which can be directly verified against manual CPI calculations.

3. System Design

3.1 Five-Stage Pipeline Architecture

The system is based on the classic 5-stage RISC-V pipeline as implemented in the Ripes simulator. Each stage performs a specific and distinct function, and each stage operates concurrently on a different instruction:

Stage	Full Name	Function
IF	Instruction Fetch	Fetches the next instruction from memory using the Program Counter (PC).
ID	Instruction Decode	Decodes the instruction and reads the required register values from the register file.
EX	Execute	Performs arithmetic/logic operations using the ALU, or computes memory addresses for load/store. Branch conditions are also evaluated here.
MEM	Memory Access	Reads from or writes to data memory. Active only for LW and SW instructions; passes through unchanged for other instruction types.
WB	Write Back	Writes the computed result back to the destination register in the register file, completing instruction execution.

Table 1: Pipeline Stages Description

3.2 Instruction Mix vs. CPI Effect

The following table summarizes how each instruction type impacts the pipeline and what CPI effect is expected:

Instruction Type	Pipeline Impact	Stall Type	CPI Effect
Arithmetic (ALU)	Executes entirely in EX stage using registers. No memory access required.	Data forwarding handles RAW hazards	Low (ideal $\approx$ 1.0)
Memory (Load/Store)	Requires MEM stage access to data memory. Load-use hazards cause pipeline stalls.	Load-use stall: 1+ cycle per dependent load	Medium (1.5–2.0)
Control (Branch/Jump)	Branch condition resolved in EX stage. Incorrectly fetched instructions flushed.	Control flush: 1–2 cycles per branch taken	High (2.0+)

Table 2: Instruction Mix vs. CPI Effect

3.3 Simulation Configuration

Parameter	Value
Simulator	Ripes — RISC-V Visual Simulator (open-source)
Processor Model	5-Stage Pipeline Processor (RISC-V 32-bit)
ISA	RISC-V RV32IMC (includes M-extension for MUL/DIV)

Hazard Handling	Data Forwarding enabled + Pipeline Stalls for unresolvable hazards
Memory Model	Separate Instruction Memory and Data Memory (Harvard architecture)
Platform	Windows / Linux / macOS

Table 3: Simulation Configuration

4. Experimental Methodology

To evaluate CPI variations across instruction types, three assembly programs were designed with clearly distinct instruction mixes: an arithmetic-heavy program, a memory-heavy program, and a control-heavy program. Each program was carefully constructed to isolate the target instruction type and minimize contamination from other instruction categories.

Each program was executed separately on the Ripes 5-stage pipeline simulator. After execution was complete, performance metrics were recorded directly from the Execution Info panel. CPI values were also manually verified using the formula:

$$\text{CPI} = \text{Total Clock Cycles} / \text{Total Instructions Retired}$$

4.1 Steps Followed

1. Designed a RISC-V assembly program targeting a specific instruction mix.
2. Entered the program into the Ripes source code editor with Assembly input type selected.
3. Selected the 5-stage pipeline processor model (RISC-V RV32IMC).
4. Assembled the program and ran it to completion using the Run/Step-through controls.
5. Recorded Total Clock Cycles and Instructions Retired from the Execution Info panel.
6. Recorded CPI and IPC values displayed by Ripes.
7. Manually verified: $\text{CPI} = \text{Total Cycles} / \text{Instructions Retired}$.
8. Noted final register values to confirm correct program execution.
9. Repeated steps 1–8 for each of the three instruction mix programs.

5. Program Descriptions and Implementation

5.1 Program 1 — Arithmetic-Heavy Instruction Mix

This program is designed to contain exclusively arithmetic and logical instructions. Four registers are first initialized with constant values using ADDI: x1=10, x2=20, x3=5, and x4=2. This is followed by twelve consecutive register-to-register operations including ADD, SUB, MUL, and DIV, forming a chained computation where each instruction typically uses results from preceding instructions.

Because all operations are register-based and no data memory is accessed, the primary pipeline challenge is data hazards. The Ripes simulator resolves most data hazards through data forwarding (bypassing), which routes results directly from the EX or MEM stage output to the input of subsequent instructions without waiting for Write Back. The few remaining unresolvable hazards are handled through pipeline stalls. The absence of branch instructions means the pipeline is never flushed, and sequential execution proceeds with high efficiency.

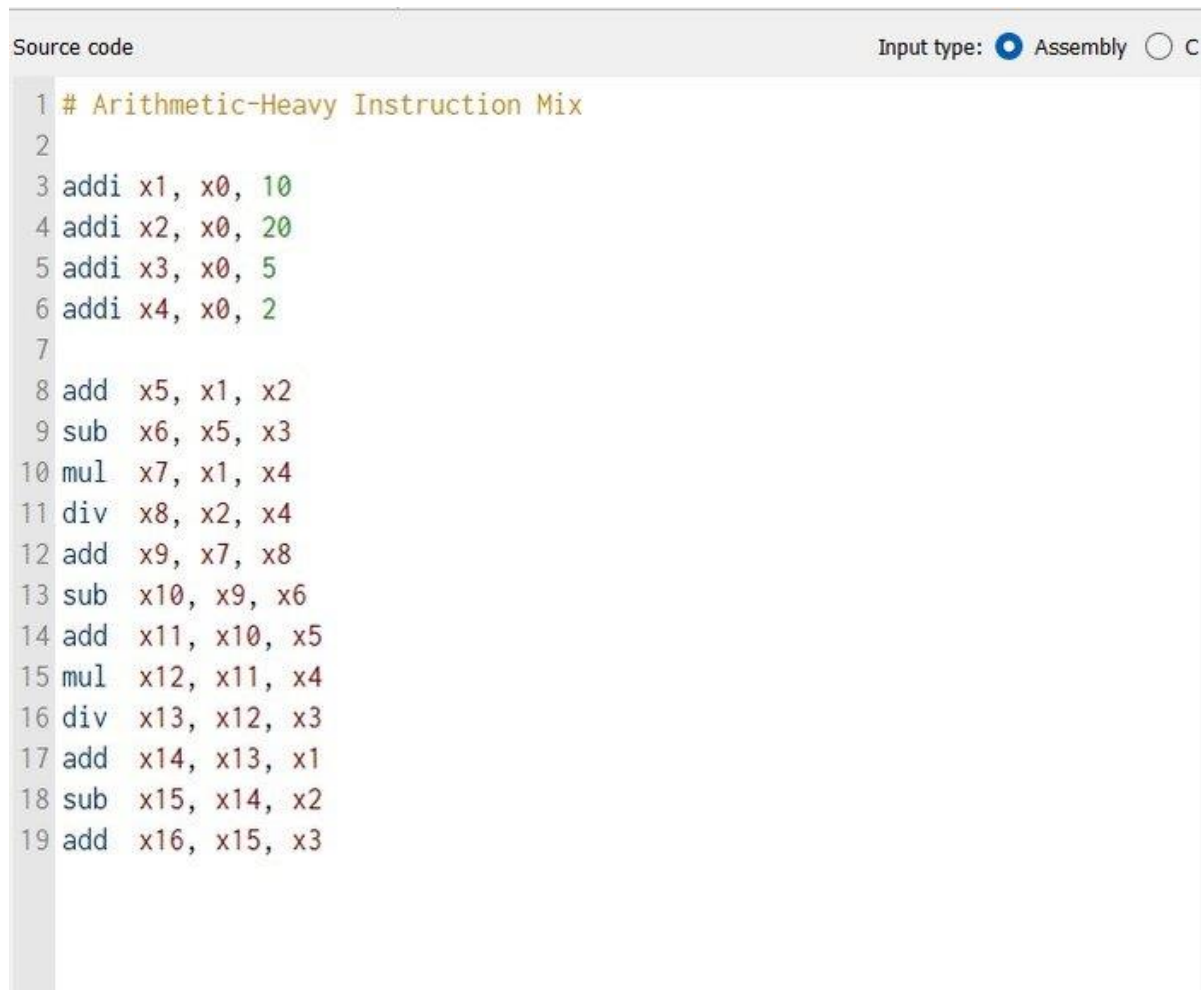
Characteristics:

- Exclusively register-to-register operations: ADDI, ADD, SUB, MUL, DIV
- No data memory access — zero LW/SW instructions
- No branch or jump instructions — completely sequential control flow
- Data forwarding handles most register dependencies without stalls
- 16 instructions executed in 20 clock cycles

Expected Behavior:

Minimal pipeline stalls, efficient ALU utilization, and the lowest CPI among all three programs. The pipeline fills quickly and remains nearly full throughout execution.

Figure 1 shows the actual RISC-V assembly source code as entered in the Ripes editor. The code initializes four registers and then performs a chain of 12 arithmetic operations:



The screenshot shows the Ripes Source Editor interface. At the top, there is a tab labeled 'Source code' and a control for 'Input type' with two radio buttons: 'Assembly' (which is selected) and 'C'. The main area contains 19 lines of RISC-V assembly code, each preceded by a line number from 1 to 19. The code is as follows:

```
1 # Arithmetic-Heavy Instruction Mix
2
3 addi x1, x0, 10
4 addi x2, x0, 20
5 addi x3, x0, 5
6 addi x4, x0, 2
7
8 add x5, x1, x2
9 sub x6, x5, x3
10 mul x7, x1, x4
11 div x8, x2, x4
12 add x9, x7, x8
13 sub x10, x9, x6
14 add x11, x10, x5
15 mul x12, x11, x4
16 div x13, x12, x3
17 add x14, x13, x1
18 sub x15, x14, x2
19 add x16, x15, x3
```

Fig 1 – RISC-V Assembly Code: Arithmetic-Heavy Instruction Mix in Ripes Source Editor

Figure 2 shows the completed Ripes simulation of the arithmetic-heavy program. The Execution Info panel (bottom centre) confirms: Cycles = 20, Instructions Retired = 16, CPI = 1.25, IPC = 0.8. The register panel (right) shows the final computed values: x16 (a6) = 0x00000009, confirming correct execution of the entire arithmetic chain:

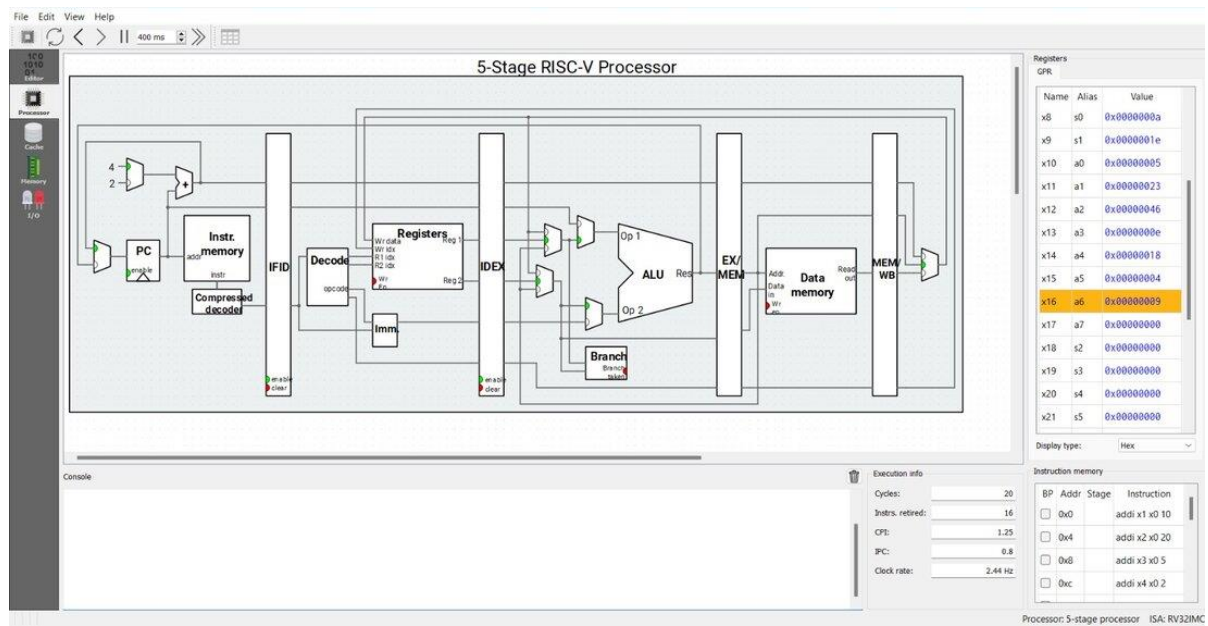


Fig 2 – Ripes Simulation: Arithmetic-Heavy Program — Cycles=20, Instrs=16, CPI=1.25, IPC=0.800

5.2 Program 2 — Memory-Heavy Instruction Mix

This program is designed to exercise the data memory subsystem of the pipeline extensively. It uses a `.data` section to define an integer array containing {5, 10, 15, 20} in memory. The program begins by loading the base address of the array using LUI and ADDI, then initializes a counter (`x2=4`) and a running sum register (`x3=0`). It then performs four interleaved sequences of LW (Load Word) followed immediately by ADD, loading each element of the array and accumulating the total. Finally, SW (Store Word) writes the computed sum back to memory.

The critical pipeline challenge in this program is the load-use data hazard: each LW instruction is immediately followed by an ADD that uses the loaded value. Since the loaded data is not available until after the MEM stage, the processor must insert a pipeline stall bubble between the LW and the dependent ADD instruction. This accounts for the increased CPI compared to the arithmetic-heavy program. Additionally, the LUI instruction itself has non-trivial execution overhead.

Characteristics:

- Uses `.data` section with array: `array .word 5, 10, 15, 20`
- Base address loaded using LUI + ADDI (hardware address setup)
- Four LW instructions each immediately followed by a dependent ADD — creates load-use hazards
- One SW instruction to store final sum result to memory
- 13 instructions executed in 21 clock cycles

Expected Behavior:

Moderate CPI due to load-use stalls. The pipeline is interrupted each time a loaded value is immediately used, but recovers between stalls. Final result `x3 = 0x00000032 = 50` decimal, which is $5+10+15+20 = 50$, confirming correct execution.

Figure 3 shows the RISC-V assembly source code for the memory-heavy program as loaded into Ripes. The `.data` section, LUI/ADDI address setup, LW/ADD interleaved sequence, and the final SW are all clearly visible:

Source code Input type: ☒ Assembly ☐ C

```

1 #Memory-heavy instruction mix
2 .data
3 array: .word 5, 10, 15, 20
4
5 .text
6
7 lui x1, 0x10000      # Load upper 20 bits of base address
8 addi x1, x1, 0       # Complete address
9
10 addi x2, x0, 4       # Count = 4
11 addi x3, x0, 0       # Sum = 0
12
13 lw x4, 0(x1)         # Load array[0]
14 add x3, x3, x4
15 lw x5, 4(x1)         # Load array[1]
16 add x3, x3, x5
17 lw x6, 8(x1)         # Load array[2]
18 add x3, x3, x6
19 lw x7, 12(x1)        # Load array[3]
20 add x3, x3, x7
21
22 sw x3, 16(x1)        # Store sum

```

Fig 3 – RISC-V Assembly Code: Memory-Heavy Instruction Mix (.data array + LW/ADD/SW pattern)

Figure 4 shows the completed Ripes simulation. The Execution Info panel confirms: Cycles = 21, Instructions Retired = 13, CPI = 1.62, IPC = 0.619. The register x3 (gp) = 0x00000032 = 50 confirms the array sum was computed correctly. The Instruction Memory panel (right) shows the program starting with lui x1 x0 0x10000, followed by the ADDI and memory access sequence:

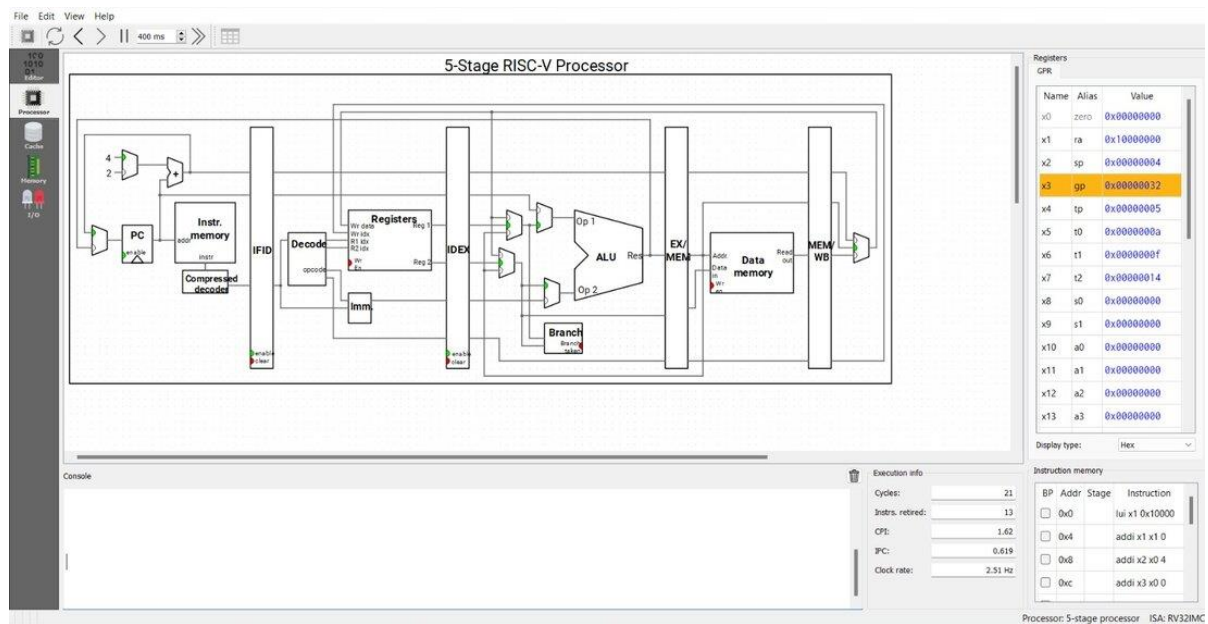


Fig 4 – Ripes Simulation: Memory-Heavy Program — Cycles=21, Instrs=13, CPI=1.62, IPC=0.619

5.3 Program 3 — Control-Heavy Instruction Mix

This program is designed to maximize control hazards by using multiple different types of conditional branch instructions in rapid succession. The program first initializes three registers: x1=0 (counter), x2=5 (limit), and x3=0 (accumulator). It then uses four different branch instructions — BEQ (Branch if Equal), BNE (Branch if Not Equal), BLT (Branch if Less Than), and BGE (Branch if Greater or Equal) — chained with labels L1, L2, L3, L4, interspersed with ADDI accumulator increments. The program concludes with a final ADD.

Each branch instruction causes a control hazard: the processor has already fetched the subsequent instruction(s) into the IF and ID stages before the branch condition is evaluated in the EX stage. When the branch is taken, those incorrectly fetched instructions must be flushed (replaced with pipeline bubbles), wasting 1–2 clock cycles per flush. With four branch instructions in a 9-instruction program, branch flushes account for a substantial fraction of total cycles, resulting in the highest CPI of all three programs.

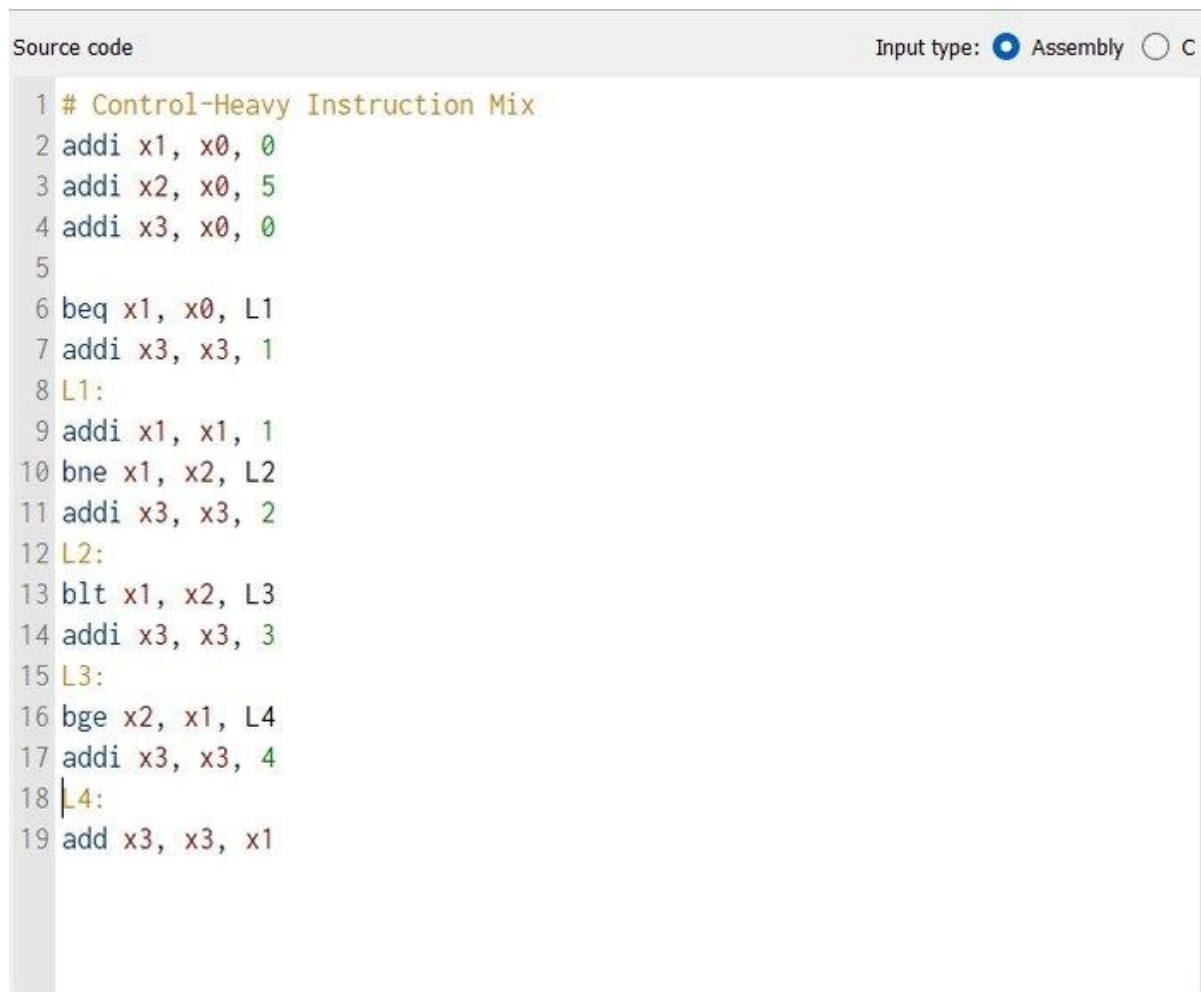
Characteristics:

- Four different branch types: BEQ, BNE, BLT, BGE
- Multiple labels: L1, L2, L3, L4 defining branch targets
- Each branch causes pipeline flush of 1–2 incorrectly fetched instructions
- Control hazards dominate pipeline behavior
- 9 instructions executed in 21 clock cycles

Expected Behavior:

Highest CPI due to frequent pipeline flushes. The pipeline cannot stay fully loaded because each branch disrupts sequential instruction fetch. Final register x3 (gp) = 0x00000001 reflects the accumulation through taken/not-taken branch paths, confirming correct execution despite the hazards.

Figure 5 shows the control-heavy assembly program source code in the Ripes editor. The BEQ, BNE, BLT, and BGE instructions and their associated labels are clearly visible:



The screenshot shows a code editor window titled "Source code". In the top right corner, there is a label "Input type:" followed by two radio buttons: "Assembly" (which is selected) and "C". The main area of the editor contains 19 lines of RISC-V assembly code. The code starts with a comment "# Control-Heavy Instruction Mix" on line 1. It then performs several additions to registers x1, x2, and x3. Line 6 contains a branch instruction "beq x1, x0, L1". Line 10 contains "bne x1, x2, L2". Line 13 contains "blt x1, x2, L3". Line 16 contains "bge x2, x1, L4". The code uses labels L1, L2, L3, and L4 to indicate jumps. The final instruction on line 19 is "add x3, x3, x1".

```
1 # Control-Heavy Instruction Mix
2 addi x1, x0, 0
3 addi x2, x0, 5
4 addi x3, x0, 0
5
6 beq x1, x0, L1
7 addi x3, x3, 1
8 L1:
9 addi x1, x1, 1
10 bne x1, x2, L2
11 addi x3, x3, 2
12 L2:
13 blt x1, x2, L3
14 addi x3, x3, 3
15 L3:
16 bge x2, x1, L4
17 addi x3, x3, 4
18 L4:
19 add x3, x3, x1
```

Fig 5 – RISC-V Assembly Code: Control-Heavy Instruction Mix (BEQ, BNE, BLT, BGE with labels L1–L4)

Figure 6 shows the completed Ripes simulation of the control-heavy program. The Execution Info panel confirms: Cycles = 21, Instructions Retired = 9, CPI = 2.33, IPC = 0.429. The register panel shows x3 (gp) = 0x00000001. The Instruction Memory panel (right) shows the program starting with addi x1 x0 0, addi x2 x0 5, addi x3 x0 0, and beq x1 x0 8, confirming the control-flow structure:

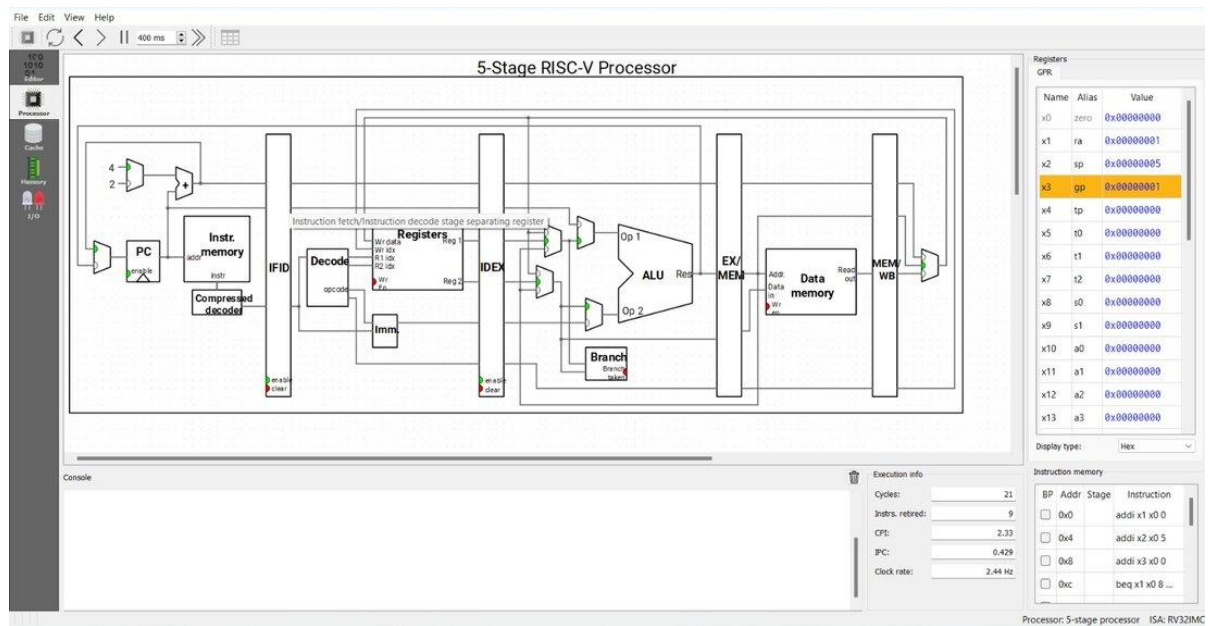


Fig 6 – Ripes Simulation: Control-Heavy Program — Cycles=21, Instrs=9, CPI=2.33, IPC=0.429

6. Experimental Results

The following table presents the complete experimental results recorded directly from the Ripes Execution Info panel for all three programs:

Program Type	Total Cycles	Instrs Retired	CPI	IPC	Final Reg
Arithmetic-Heavy	20	16	1.25	0.800	x16=0x9
Memory-Heavy	21	13	1.62	0.619	x3=0x32
Control-Heavy	21	9	2.33	0.429	x3=0x1

Table 4: Experimental Results — CPI Comparison Across All Three Instruction Mix Programs

6.1 Manual CPI Verification

All CPI values were manually verified using the formula $CPI = \text{Total Cycles} / \text{Instructions Retired}$:

Arithmetic-Heavy: $CPI = 20 / 16 = 1.25$ ✓ matches Ripes output

Memory-Heavy: $CPI = 21 / 13 \approx 1.615 \approx 1.62$ ✓ matches Ripes output

Control-Heavy: $CPI = 21 / 9 \approx 2.333 \approx 2.33$ ✓ matches Ripes output

6.2 Detailed Performance Metrics

Metric	Arithmetic	Memory	Control	Observation
Total Clock Cycles	20	21	21	Memory & Control use same total cycles; far fewer instructions retired in Control
Instructions Retired	16	13	9	Fewer instructions in same cycles = higher CPI = more wasted cycles
CPI	1.25	1.62	2.33	Direct measure of pipeline efficiency per instruction
IPC	0.800	0.619	0.429	Inverse of CPI; higher = better throughput
% of Ideal CPI (1.0)	80% efficiency	62% efficiency	43% efficiency	Control-heavy wastes 57% of potential throughput
Primary Hazard	RAW (forwarded)	Load-use stalls	Branch flushes	Control hazards cause the most severe degradation

Table 5: Detailed Performance Metrics from Ripes Simulation

7. Result Analysis and Discussion

7.1 Arithmetic Instruction Performance (CPI = 1.25, IPC = 0.800)

The arithmetic program achieved the best CPI of 1.25 among all three programs, corresponding to a pipeline efficiency of 80% ($1.0/1.25$). This result is expected because all ADD, SUB, MUL, DIV, and ADDI instructions operate exclusively on registers and complete their work in the EX stage. No data needs to travel to or from memory, eliminating load-use hazards entirely.

The small deviation from the ideal CPI of 1.0 is attributable to RAW (Read After Write) data hazards in the chained computation sequence. For example, when add x5, x1, x2 is followed immediately by sub x6, x5, x3, the value of x5 is not yet written back when sub x6 needs to read it. The Ripes simulator resolves most such hazards through data forwarding, which routes the result from the EX stage output directly to the input of the next instruction, avoiding a full stall. The few remaining cases where forwarding cannot fully resolve the dependency result in the observed CPI slightly above 1.0.

The final register value x16 (a6) = 0x00000009 = 9 decimal confirms that the entire arithmetic computation chain executed correctly despite the data dependencies.

7.2 Memory Instruction Performance (CPI = 1.62, IPC = 0.619)

The memory program showed a 30% higher CPI than the arithmetic program (1.62 vs 1.25), reducing pipeline efficiency to 62%. This degradation is caused by load-use data hazards: each of the four LW instructions is immediately followed by an ADD instruction that uses the loaded value. Since the loaded data arrives at the register file only after the MEM stage completes, the pipeline must insert a one-cycle stall bubble between the LW and the dependent ADD. With four such load-use pairs in the program, this accounts for the majority of extra cycles above CPI 1.0.

Additionally, the LUI instruction at the start of the program — used to load the upper 20 bits of the base address — contributes to pipeline filling overhead. The SW (Store Word) instruction at the end also uses the MEM stage, though since no subsequent instruction depends on the stored value, no stall is needed there.

The final register x3 (gp) = 0x00000032 = 50 decimal verifies that all four loads ($5+10+15+20 = 50$) were correctly accumulated, confirming the simulator handled load-use hazards correctly through hardware stall insertion while still producing the right result.

7.3 Control Instruction Performance (CPI = 2.33, IPC = 0.429)

The control-heavy program exhibited the worst performance with CPI = 2.33, representing only 43% pipeline efficiency — more than twice the CPI of the arithmetic program. This severe degradation is caused by control hazards from the four branch instructions (BEQ, BNE, BLT, BGE). Each time one of these branch instructions is in the EX stage and the branch condition is evaluated, the processor has already fetched subsequent instructions into IF and ID. If the branch is taken, those instructions must be flushed from the pipeline, introducing 1-2 wasted cycles per branch.

With 4 branch instructions across 9 total retired instructions, branches represent 44% of the instruction stream. Each taken branch contributes at minimum one flush cycle to the cycle

count. With 21 total cycles for 9 instructions versus the theoretical minimum of approximately $9 + 4$ (pipeline fill) = 13 cycles, the difference of 8 extra cycles is attributable to branch flush penalties.

The final register x3 (gp) = 0x00000001 confirms correct program execution: despite multiple pipeline flushes, the processor correctly tracked which instructions should be committed and which should be discarded, and produced the right final accumulator value.

7.4 Hazard Type Summary

Aspect	Arithmetic	Memory	Control
Dominant hazard	RAW (data forwarded)	Load-use stall	Branch flush
Stall mechanism	Data forwarding (no stall)	1-cycle bubble per LW	1–2 cycle flush per branch
Extra cycles vs ideal	~4 extra cycles	~8 extra cycles	~12 extra cycles
Result correctness	x16=0x9 ✓	x3=50 (0x32) ✓	x3=1 (0x1) ✓

Table 6: Hazard Types and Their Impact on Each Instruction Mix Program

8. CPI Comparison

Arithmetic CPI < Memory CPI < Control CPI

1.25 < 1.62 < 2.33

This confirmed ordering demonstrates that instruction mix is a critical determinant of processor efficiency. The control-heavy program is 86.4% slower than the arithmetic-heavy program in terms of CPI (2.33 vs 1.25), and the memory-heavy program is 29.6% slower (1.62 vs 1.25).

8.1 CPI Bar Representation

The following proportional bar chart representation compares CPI values across all three programs. Longer bars indicate higher CPI and lower pipeline efficiency:



8.2 Graph Interpretation

- The Arithmetic-Heavy program shows the shortest bar, confirming efficient pipeline usage with near-ideal throughput. Data forwarding resolves most hazards without stalling the pipeline.
- The Memory-Heavy program shows a moderate increase due to load-use stalls inserted when loaded values are immediately consumed by the next instruction. Each load-use pair costs one extra cycle.
- The Control-Heavy program shows the longest bar by a wide margin, demonstrating that branch instructions and pipeline flushes cause the most severe performance degradation. Each taken branch discards 1-2 already-fetched instructions.

These results confirm both the theoretical predictions from computer architecture literature and the practical measurements obtained from the Ripes simulation environment.

9. Advantages of the Experiment

- Provides clear, measurable evidence of how different instruction types impact processor performance in a real simulation environment.
- Demonstrates the practical impact of all three types of pipeline hazards — structural (avoided by design), data (RAW, forwarded), and control (branch flushes) — through direct measurement.
- Uses real simulation data rather than theoretical estimates, making results credible and verifiable.
- The Ripes visual simulator allows instruction-by-instruction pipeline visualization, enabling deeper understanding of exactly when and why stalls and flushes occur.
- Results are useful for processor design optimization: they quantify the performance cost of each hazard type and motivate architectural features like branch prediction and deeper data forwarding networks.
- Assists in compiler instruction scheduling: by knowing which instruction sequences cause stalls, compilers can reorder instructions to reduce CPI without changing program semantics.
- Provides a strong practical foundation for understanding modern superscalar and out-of-order processors, where these same hazard types occur at larger scale.

10. Limitations

- Simulation results may differ from real hardware behaviour due to the simplified memory model (no cache hierarchy) and idealized forwarding logic in Ripes.
- The programs used are small (9–16 instructions), which means the CPI is influenced by pipeline fill-up overhead that would diminish in longer programs.
- The Ripes simulator does not model branch prediction; all branches pay the full flush penalty when taken, whereas real processors with branch prediction reduce this penalty significantly.
- Structural hazards are not demonstrated in this project because the Ripes 5-stage pipeline uses separate instruction and data memories, eliminating resource conflicts by design.
- The experiment measures steady-state CPI at program end rather than cycle-by-cycle variation, masking transient pipeline effects.
- Memory hierarchy effects such as cache misses, which would significantly increase CPI in real programs, are not modelled.

11. Applications

- **CPU Architecture Research:** Provides foundational insights for designing efficient processor pipelines. Quantifying the cost of each hazard type directly informs decisions about forwarding network depth, branch prediction hardware, and pipeline stage count.
- **Processor Design Education:** Enables students to visually understand instruction flow through pipeline stages and see the real-time performance impact of hazards through direct simulation.
- **Compiler Optimization:** Compilers use knowledge of pipeline hazards to reorder instructions, insert NOPs strategically, and schedule loads early to hide latency — all aimed at reducing effective CPI.
- **Embedded Systems Design:** Understanding CPI and pipeline hazard costs is critical for real-time embedded systems where worst-case execution time (WCET) must be precisely bounded.
- **Hardware Verification:** Engineers use pipeline simulation tools like Ripes to verify hazard detection logic, forwarding paths, and flush mechanisms before committing to physical chip manufacture.
- **Computer Architecture Courses:** CPI measurement experiments form the core practical component of undergraduate computer architecture courses worldwide, bridging theory and practice.
- **Performance Profiling:** The methodology of isolating instruction mix categories and measuring CPI directly translates to real-world performance profiling on production hardware.

12. Future Enhancements

- **Advanced Branch Prediction:** Implementing a 2-bit saturating counter or branch target buffer (BTB) in Ripes to measure how prediction accuracy reduces control hazard penalties from 2 cycles toward zero cycles.
- **Data Forwarding Analysis:** Disabling forwarding and re-running the arithmetic program to explicitly measure how many extra stall cycles data forwarding eliminates, quantifying the hardware benefit.
- **Larger Program Analysis:** Executing programs with hundreds or thousands of instructions to obtain statistically robust CPI measurements free of pipeline fill-up artefacts.
- **Mixed Instruction Mix:** Designing programs with realistic blends of all three instruction types (approximating SPEC CPU benchmark instruction mixes) to study CPI under practical workloads.
- **Superscalar Pipeline:** Extending the analysis to a 2-way or 4-way superscalar pipeline to study how instruction mix and hazards compound at higher levels of instruction-level parallelism.
- **Cache Integration:** Adding L1 instruction and data cache simulation to measure the combined impact of cache misses and pipeline hazards on overall CPI, revealing the full memory hierarchy effect.
- **Out-of-Order Execution:** Studying Tomasulo's algorithm and reservation stations to understand how modern out-of-order processors dynamically reorder instructions to avoid data hazards.
- **FPGA Implementation:** Implementing the 5-stage RISC-V pipeline with hazard detection and forwarding in Verilog or VHDL on an FPGA as a hardware realization of the concepts studied.

13. Conclusion

This project successfully evaluated how different instruction mixes influence processor performance in a 5-stage pipelined RISC-V architecture using the Ripes visual simulator. Three custom assembly programs — each targeting a distinct instruction category — were designed, executed, and analyzed with all performance metrics captured directly from the simulator.

The arithmetic-heavy program achieved the best performance with $CPI = 1.25$ and $IPC = 0.800$, demonstrating 80% pipeline efficiency. The absence of memory accesses and branch instructions allowed the pipeline to remain nearly full throughout execution, with data forwarding resolving the few register dependencies that arose. Final register $x16 = 0x9$ confirmed correct execution.

The memory-heavy program showed moderate performance degradation with $CPI = 1.62$ and $IPC = 0.619$, representing 62% pipeline efficiency. Load-use hazards — arising from the pattern of loading array elements and immediately accumulating them — forced the pipeline to insert stall bubbles between dependent instructions. Despite these delays, the correct sum of $5+10+15+20 = 50$ ($x3 = 0x32$) was produced.

The control-heavy program exhibited the worst performance with $CPI = 2.33$ and $IPC = 0.429$, representing only 43% pipeline efficiency. Frequent conditional branch instructions (BEQ, BNE, BLT, BGE) caused repeated pipeline flushes, discarding incorrectly-fetched instructions and wasting clock cycles. With 4 branch instructions in a 9-instruction program, branch penalties dominated the cycle count.

The confirmed ordering — Arithmetic $CPI (1.25) < Memory\ CPI (1.62) < Control\ CPI (2.33)$ — provides clear quantitative evidence that instruction mix is a primary determinant of processor efficiency. Optimizing instruction distribution through compiler scheduling, branch prediction, data forwarding, and load scheduling can significantly enhance system performance and brings real processors closer to the theoretical ideal of $CPI = 1.0$.

14. References

10. Patterson, D. A., & Hennessy, J. L. (2020). Computer Organization and Design RISC-V Edition: The Hardware Software Interface (2nd ed.). Morgan Kaufmann.
11. Hennessy, J. L., & Patterson, D. A. (2019). Computer Architecture: A Quantitative Approach (6th ed.). Morgan Kaufmann.
12. Waterman, A., & Asanovic, K. (2019). The RISC-V Instruction Set Manual Volume I: Unprivileged ISA. RISC-V Foundation.
13. Ripes RISC-V Visual Simulator (2024). GitHub Repository. Available at: <https://github.com/mortbopet/Ripes>
14. RISC-V International. (2024). RISC-V Technical Specifications. Available at: <https://riscv.org/technical/specifications/>
15. Tomasulo, R. M. (1967). An efficient algorithm for exploiting multiple arithmetic units. IBM Journal of Research and Development, 11(1), 25–33.
16. Standard Computer Organization and Architecture textbooks — as referenced in APJ Abdul Kalam Technological University B.Tech curriculum.